

PLAN DE MODIFICACION DEL PROTOTIPO

Dungeon / DnD -> RPG tactico orientado a preparacion

Objetivo del documento: indicar a un agente de desarrollo en VS Code que debe inspeccionar el repositorio actual y modificarlo incrementalmente hasta obtener un prototipo funcional y jugable del nuevo bucle: **Preparacion -> Combate automatico -> Diagnostico -> Reajuste -> Reintento.**

Principio rector: no reescribir por reescribir. El prototipo existente debe tratarse como base del motor de combate. Se conserva todo componente util de estadísticas, dano, enemigos, experiencia, armas y flujo de encuentro; solo se reemplaza o encapsula lo necesario para soportar party, IA, registro causal y diagnostico.

Fuente de diseño: GDD de Arquitectura Base y GDD Tecnico de Prototipo. Esta especificacion reduce deliberadamente el alcance para validar primero el bucle jugable antes de implementar energia, monetizacion, reclutamiento completo, mapa o contenido narrativo amplio.

Estado objetivo	Cantidad minima
Personajes jugables	3
Configuraciones/builds por personaje	2
Armas funcionalmente distintas	3
Jefe de prueba	1 con 2 mecanicas
Combate	Automatico con prioridades simples
Diagnostico	1 causa dominante o desgaste general
Reintento	Permitido tras modificar preparacion

1. Resultado funcional esperado

Al finalizar esta modificacion, el proyecto debe permitir una sesion completa sin editar datos manualmente ni reiniciar el programa entre pasos. El jugador debe poder:

- Seleccionar una party de 3 personajes disponibles.
- Elegir una build/configuracion y un arma para cada personaje.
- Iniciar un combate contra un jefe de prueba.
- Observar como los personajes y el jefe actuan automaticamente segun velocidad y prioridades.
- Perder de forma posible y comprensible ante una de las mecanicas del jefe.
- Recibir un diagnostico post-combate basado en el Combat Log.
- Cambiar build, arma o prioridad de IA antes del reintento.
- Volver a combatir y poder ganar como consecuencia de una mejor preparacion.

El prototipo se considera valido aunque la interfaz sea de consola o una UI minima. La prioridad es demostrar que la preparacion cambia el resultado y que el diagnostico ayuda a identificar que cambiar.

2. Regla de trabajo para el agente

El agente **NO debe asumir** nombres de archivos, clases o carpetas que no existan. Primero debe mapear el repositorio y adaptar este plan a la estructura real.

Secuencia obligatoria:

Paso	Accion del agente	Entregable
0.1	Inspeccionar arbol del repositorio, entry point, modulos de combate, modelos de personaje, enemigos, armas, progresion y UI/consola.	Mapa breve de archivos y responsabilidades.
0.2	Identificar codigo reutilizable y codigo acoplado a menus/UI.	Lista: conservar / encapsular / reemplazar.
0.3	Ejecutar el proyecto y documentar el flujo actual.	Comando de ejecucion y estado baseline.
0.4	Crear una rama o checkpoint antes de cambios invasivos.	Punto de retorno verificable.
0.5	Implementar por fases, ejecutando pruebas despues de cada una.	Cambios pequeños y verificables.

Prohibido: rehacer el proyecto completo, cambiar de motor/framework sin necesidad, introducir sistemas de energia/monetizacion, construir un arbol de talentos grande o desarrollar contenido narrativo antes de validar el bucle.

3. Arquitectura objetivo minima

La logica debe quedar separada de la presentacion. El agente puede reutilizar clases existentes, pero al final deben existir responsabilidades equivalentes a las siguientes:

Componente	Responsabilidad	Puede reutilizar
Character / CharacterData	Stats, nivel, arma, build, estados, cooldowns.	Modelo actual de personaje.
Weapon / ItemData	Bonos y propiedades tacticas del arma.	Diccionarios/clases de armas existentes.
Encounter / BossData	Stats del jefe, fases y mecanicas.	Modelo actual de enemigo.
CombatResolver	Orden de turnos, eleccion de accion, dano, estados y fin de combate.	Funciones actuales de combate.
AIController / PriorityResolver	Decidir accion automatica por reglas configuradas.	Nueva capa.
CombatLog	Registrar eventos estructurados, no solo texto.	Logs actuales si existen.
DiagnosticEngine	Calcular causa dominante de derrota.	Nuevo.
BuildManager	Aplicar una de dos configuraciones por personaje.	Stats/equipo existentes.
GameLoop / PrototypeController	Preparacion -> combate -> diagnostico -> reintento.	Loop/menu principal actual.
UI / Console Layer	Mostrar opciones y resultados. No contiene reglas de combate.	UI/console actual.

Dependencia recomendada:

```
UI / Console
|
PrototypeController
|
+--> BuildManager
|
+--> CombatResolver  ----> CombatLog
|                       |
|                       +--> DiagnosticEngine
+--> AIController
+--> CharacterData / EncounterData
```

La UI puede leer resultados de los modulos de logica, pero las formulas no deben depender de widgets, input() o escenas concretas.

4. Contratos de datos minimos

El agente debe adaptar los modelos actuales, no reemplazarlos si ya contienen la informacion. Como minimo, cada personaje debe poder representar:

```
Character
- id / nombre
- hp_max / hp_actual
- ataque
- defensa
- velocidad
- critico_pct (opcional en primera iteracion)
- rol_tactico
- arma_equipada
- build_activa
- estados_activos[]
- prioridades_ia[]
- habilidades[]
```

Cada arma debe dejar de ser solo un numero de dano. Para el prototipo, usar tres identidades tacticas claras:

Arma	Funcion prototipo	Tag/efecto sugerido
Daga o arma rapida	Presion sobre objetivo vulnerable; critico/velocidad.	single_target / critico
Martillo o arma pesada	Ruptura de escudo.	shield_break; factor alto contra escudo
Lanza / machete / tercera arma	Dano sostenido o respuesta a estados.	bleed / anti_bleed / perforacion

El nombre exacto de las armas debe respetar el contenido actual del proyecto cuando sea posible. Lo importante es que cada una resuelva un problema distinto.

Encuentro de prueba:

```
BossEncounter
- hp_max
- ataque
- defensa
- velocidad
- escudo_max
- mecanica_1: sangrado acumulativo
- mecanica_2: ventana de ruptura de escudo
- tag_contador_1: anti_sangrado
- tag_contador_2: anti_escudo
```

5. Cambio del sistema de combate

El cambio central es pasar de una seleccion manual continua de acciones a una simulacion automatica gobernada por preparacion. El combate debe seguir siendo determinista en estructura, con aleatoriedad limitada a critico/iniciativa si ya existe.

Flujo de una ronda:

1. Recalcular iniciativa de todos los actores vivos.
2. Ordenar de mayor a menor iniciativa.
3. Para cada actor vivo:
 - a. Resolver estados al inicio del turno.
 - b. Elegir accion con AIController.
 - c. Validar objetivo y coste/cooldown.
 - d. Ejecutar accion.
 - e. Registrar eventos estructurados.
 - f. Verificar muertes / cambio de fase / fin de combate.
4. Reducir duraciones y cooldowns.
5. Repetir hasta victoria, derrota o limite de seguridad.

Formula base sugerida, si la existente no es equivalente:

```
dano_bruto = ataque_atacante * multiplicador_habilidad
factor_mitigacion = 100 / (100 + defensa_objetivo)
dano_final = dano_bruto * factor_mitigacion * modificadores
```

No es obligatorio sustituir una formula existente que ya este balanceada. Si el proyecto actual tiene una formula de dano funcional, encapsularla en CombatResolver y mantenerla durante la primera migracion.

Iniciativa sugerida:

```
iniciativa = velocidad + random(0, velocidad * 0.10)
```

Recalcular por ronda permite que buffs/debuffs de velocidad sean utiles mas adelante.

6. IA automatica configurable

Para el prototipo no construir un sistema visual complejo de programacion de IA. Cada personaje necesita una lista ordenada de reglas. El agente debe implementar un evaluador simple de primera coincidencia.

```
def elegir_accion(actor, aliados, enemigos, contexto):
    for regla in actor.prioridades_ia:
        accion = regla.evaluar(actor, aliados, enemigos, contexto)
        if accion is not None:
            return accion
    return ataque_basico(actor, seleccionar_objetivo(enemigos))
```

Reglas minimas recomendadas:

Prioridad	Condicion	Accion
1	Aliado por debajo de umbral y actor puede curar.	Curar.
2	Jefe tiene escudo activo y actor posee herramienta de ruptura.	Usar ruptura.
3	Habilidad unica disponible y condicion valida.	Usar habilidad unica.
4	Existe objetivo vulnerable/prioritario.	Ataque especializado.
5	Ninguna regla anterior.	Ataque basico.

Para el primer vertical slice basta con que el jugador pueda elegir entre dos presets de prioridad por personaje. La edicion granular de reglas puede posponerse.

7. Combat Log estructurado

Todo evento relevante debe quedar registrado como datos. Un print() puede seguir mostrando el combate, pero no puede ser la unica fuente del diagnostico.

```
CombatEvent {
    turno: int,
    tipo: str,                # dano_recibido, dano_infligido, muerte,
                             # escudo_no_roto, estado_aplicado, curacion...
    actor_id: str | None,
    objetivo_id: str | None,
    fuente_tag: str | None, # sangrado, escudo, fisico, etc.
    cantidad: float | None,
    hp_restante_pct: float | None,
    metadata: dict
}
```

Eventos obligatorios para que el prototipo pueda diagnosticar:

- Cada instancia de dano recibido por la party, con su fuente_tag.
- Aplicacion y ticks de sangrado.
- Activacion del escudo del jefe.
- Ruptura exitosa del escudo.
- Fallo de ruptura dentro de la ventana y posterior ataque de castigo.
- Muerte de cada miembro de la party.
- Victoria o derrota final.

8. DiagnosticEngine

En derrota, el motor debe calcular una causa dominante. No debe inventar una recomendacion si ninguna causa explica suficientemente el resultado.

```
def analizar_derrota(combat_log):
    contribuciones = {}

    for evento in combat_log:
        if evento.tipo in ("dano_recibido", "muerte", "castigo_escudo"):
            tag = normalizar_causa(evento.fuente_tag)
            contribuciones[tag] = contribuciones.get(tag, 0) + peso(evento)

    total = sum(contribuciones.values())
    if total <= 0:
        return Diagnostico("desgaste_general", 0)

    causa, valor = max(contribuciones.items(), key=lambda x: x[1])
    pct = valor / total * 100

    if pct < 40:
        return Diagnostico("desgaste_general", pct)

    return Diagnostico(causa, pct)
```

Mapeo minimo de causa -> sugerencia:

Causa	Mensaje	Cambio sugerido
sangrado	El dano de sangrado fue la principal causa de derrota.	Usar build/arma/efecto anti-sangrado.
escudo_no_roto	La party no rompio el escudo dentro de la ventana.	Equipar arma de ruptura o priorizar shield_break.
dano_directo	La party no soporta el dano base del encuentro.	Aumentar defensa/vida o revisar nivel.
desgaste_general	No hubo una causa dominante clara.	Revisar nivel, stats y preparacion general.

El diagnostico debe incluir el porcentaje aproximado que motivo la conclusion, para que durante desarrollo podamos comprobar si el motor esta siendo razonable.

9. Builds: simplificacion deliberada

No implementar todavia el arbol de 8-10 nodos por personaje. Para validar el concepto basta con dos builds predefinidas por personaje. Deben ser mutuamente relevantes, no una claramente superior.

Build	Ejemplo de efectos	Proposito
A - Ofensiva	+20% ataque, +critico o velocidad.	Ganar rapido cuando no existe counter especifico.
B - Adaptacion	+ruptura de escudo y/o resistencia a sangrado; menor DPS bruto.	Resolver mecanicas del jefe.

Implementacion sugerida:

```
BuildPreset {
  id: str,
  nombre: str,
  stat_modifiers: dict,
  tags: set[str],
  ai_priority_override: list | None
}

def aplicar_build(character, build):
  # recomponer stats desde base + equipo + build
  # no acumular modificadores al cambiar varias veces
```

Importante: al cambiar de build debe recalcularse desde stats base. Evitar sumar modificadores repetidamente sobre el estado ya modificado.

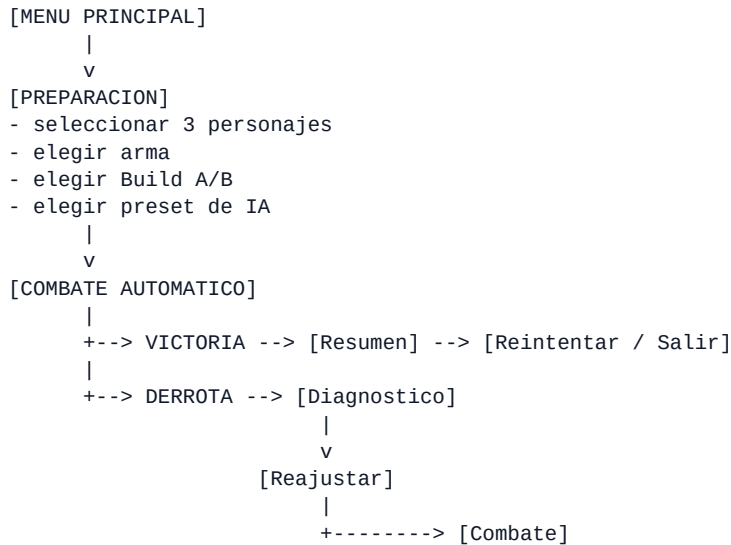
10. Jefe de prueba y condiciones tacticas

El jefe debe demostrar que una preparacion equivocada puede perder y una correcta puede ganar sin aumentar artificialmente el nivel entre intentos.

Fase	Mecanica	Respuesta esperada
Inicio	Sangrado acumulativo o periodico.	Resistencia/limpieza/mitigacion anti-sangrado.
50% HP aprox.	Activa escudo con ventana de ruptura.	Arma o habilidad con shield_break y prioridad de IA adecuada.
Fallo de ruptura	Ataque de castigo de alto dano.	Debe ser una fuente visible en Combat Log.

Los valores concretos deben ajustarse con simulaciones hasta lograr una condicion importante: la party ofensiva debe tener una probabilidad razonable de perder, mientras que la party adaptada debe mejorar claramente sus resultados sin convertirse en victoria garantizada por un solo modificador.

11. Bucle jugable del prototipo



Pantallas o estados minimos:

- Preparacion: mostrar stats finales, arma, build y prioridad seleccionada.
- Combate: mostrar turnos/eventos importantes y HP/escudo actual.
- Resultado: victoria/derrota, numero de turnos y supervivientes.
- Diagnostico: causa, porcentaje y sugerencia concreta.
- Reajuste: volver a preparacion conservando las selecciones anteriores.

12. Plan de implementacion por fases

Fase	Objetivo	Criterio para avanzar
1	Mapeo del repo + encapsular combate existente.	El juego sigue ejecutando el combate actual sin regresiones.
2	Party de 3 + turnos automaticos + iniciativa.	Un combate 3v1 termina sin input por turno.
3	Builds/armas tacticas + jefe con 2 mecanicas.	Cambiar preparacion altera resultados observables.
4	Combat Log estructurado.	Todos los eventos criticos aparecen en el log de datos.
5	DiagnosticEngine.	Una derrota por sangrado y una por escudo producen diagnosticos distintos.
6	Loop de preparacion/reintento.	Se puede completar el ciclo entero sin reiniciar el programa.
7	Balance y limpieza.	El prototipo es estable y las pruebas de aceptacion pasan.

13. Pruebas de aceptacion obligatorias

El agente no debe declarar terminada la migracion hasta demostrar al menos los siguientes casos:

ID	Prueba	Resultado esperado
T01	Ejecutar combate 3 personajes vs jefe sin input durante los turnos.	Finaliza en victoria o derrota sin bloqueo.
T02	Equipar configuracion ofensiva contra jefe con escudo.	El escudo puede fallar en romperse y se registra castigo.
T03	Equipar configuracion de ruptura.	Aumenta de forma clara la probabilidad/consistencia de romper escudo.
T04	Provocar derrota dominada por sangrado.	Diagnostico = sangrado con contribucion $\geq 40\%$.
T05	Provocar derrota por factores mezclados.	Diagnostico = desgaste_general si ninguna causa supera 40%.
T06	Cambiar build varias veces.	Stats no se acumulan incorrectamente.
T07	Reintentar despues de derrota.	No quedan HP, estados, cooldowns o logs residuales del intento previo.
T08	Victoria tras reajuste.	El loop permite volver a preparacion y terminar el jefe.
T09	Simular multiples combates con semilla controlada.	Mismos datos + misma seed producen resultados reproducibles.

Si ya existe un framework de tests, usarlo. Si no existe, crear un script de smoke tests o tests unitarios minimos que ejecuten estas condiciones sin depender de la UI.

14. Instrumentacion de balance

Agregar un modo de simulacion que permita ejecutar N combates sin render/UI. No es necesario optimizarlo en esta etapa. Debe reportar:

- Tasa de victoria por build.
- Turnos promedio hasta victoria/derrota.
- Porcentaje de derrotas por causa diagnostica.
- Frecuencia de ruptura exitosa del escudo.
- Dano recibido por tag.
- Supervivencia promedio de cada personaje.

```
simular(encuentro, party, n=1000, seed_base=1234) -> BalanceReport
```

Meta inicial orientativa: la configuracion adaptada debe superar de manera clara a la ofensiva frente al jefe especifico. No fijar porcentajes finales hasta observar el comportamiento del sistema real.

15. Sistemas fuera de alcance

Estos sistemas aparecen en los GDD, pero **NO forman parte de esta migración**. No deben bloquear el prototipo:

Sistema	Decision
Energía / regeneración idle	Posponer.
Mazmorras secundarias completas	Posponer; simular el acceso al counter mediante builds/armas disponibles.
Reclutamiento táctico y reputación	Posponer. Usar roster fijo de 3 personajes.
Árbol de talentos completo	Posponer. Sustituir temporalmente por 2 builds por personaje.
Economía, monetización, pase	Excluir.
Mapa y exploración	Excluir de este vertical slice.
Narrativa extensa	Excluir; solo textos funcionales del encuentro.
Arte final / animaciones complejas	Excluir. UI provisional es aceptable.

La razón es metodológica: primero debe demostrarse que perder -> entender -> preparar -> ganar es divertido y legible. El resto se construye sobre esa validación.

16. Criterio de terminado

El agente puede considerar terminado el prototipo cuando cumpla simultáneamente:

- El proyecto arranca con el procedimiento documentado y sin errores.
- Existe una party de 3 personajes y un jefe de prueba.
- El combate es automático y usa iniciativa/prioridades.
- Las armas/builds tienen identidades tácticas distintas.
- El jefe usa sangrado y escudo/ruptura.
- El Combat Log registra causas en forma estructurada.
- DiagnosticEngine identifica sangrado, fallo de escudo o desgaste general.
- El jugador puede modificar preparación y reintentar sin reiniciar.
- Existe al menos un escenario reproducible donde la preparación incorrecta pierde y una correcta mejora claramente el resultado.
- Las pruebas T01-T09 pasan o quedan documentadas con cualquier excepción técnica real.

17. Instruccion operativa para pegar al agente de VS Code

El siguiente bloque puede utilizarse como instruccion inicial del agente junto con este PDF:

Lee este PDF completo antes de modificar codigo.

1. Inspecciona primero el repositorio y localiza el entry point, combate, personaje, enemigos, armas, progresion y UI/console.
2. Resume que archivos vas a reutilizar, cuales encapsular y cuales crear.
3. No hagas una reescritura total. Conserva el comportamiento actual mientras introduces la nueva arquitectura por fases.
4. Implementa solo el vertical slice definido aqui: party de 3, 2 builds por personaje, 3 armas tacticas, 1 jefe con sangrado y escudo, combate automatico, Combat Log, DiagnosticEngine y loop de reintento.
5. No implementes energia, monetizacion, reclutamiento, mapa, narrativa extensa ni arbol de talentos completo.
6. Despues de cada fase ejecuta el proyecto/tests y corrige regresiones antes de seguir.
7. Si una instruccion de este PDF contradice la estructura real del repositorio, adapta la implementacion conservando la responsabilidad funcional descrita; no inventes archivos o APIs solo para coincidir con los nombres del documento.
8. Al finalizar, entrega un resumen de archivos modificados, decisiones tomadas, pruebas ejecutadas y pendientes reales.

Fin de la especificacion.